NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY
Faculty of Computing and Informatics
School of Computing
Department of Software Engineering

13 Jackson Kaujeua Street
Private Bag 13388
Windhoek
NAMIBIA

T: +264 61 207 2052
F: +264 61 207 9052
E: dse@nust.na
W: www.nust.na

PAP521S – Programming In Practice

Week 1: Introduction to Programming in C (13 – 17 July 2026)

Learning Outcomes

At the end of this lesson, students should be able to:

- Explain what computer programming is.
- Describe the program compilation process.
- Explain the purpose of the GCC compiler.
- Differentiate between source code and executable files.
- Explain the purpose of an Integrated Development Environment (IDE).
- Identify the components of a simple C program.
- Write, compile and execute their first C program.
- Use the printf() and scanf() functions.
- Configure Visual Studio Code for C programming.
- Create a GitHub account and upload their first C project.

1. Introduction

What is Programming?

Programming is the process of designing and writing instructions that tell a computer how to perform specific tasks.

A computer cannot think for itself. It only understands machine language (0s and 1s). Therefore, programmers write programs using programming languages such as:

- C
- C++
- Java
- Python
- C#
- JavaScript

These languages are then translated into machine language so that the computer can execute them.

Real-life Example

Suppose you want a municipality to calculate employees' salaries every month.

Instead of calculating each salary manually, we can write a computer program that:

- Reads employee details
- Calculates salaries
- Calculates taxes
- Produces payslips

The computer performs these calculations quickly and accurately.

Why Learn Programming?

Programming enables us to:

- Solve real-world problems.
- Automate repetitive tasks.
- Develop software applications.
- Build websites.
- Develop mobile applications.
- Create games.
- Develop Artificial Intelligence systems.
- Build Enterprise Resource Planning (ERP) systems.

Programming Languages

Programming languages can generally be classified into:

Low-Level Languages

- Machine Language
- Assembly Language

Advantages

- Very fast
- Direct hardware control

Disadvantages

- Difficult to write
- Difficult to understand

High-Level Languages

Examples

- C
- Java

- Python
- C#
- JavaScript

Advantages

- Easier to read
- Easier to debug
- Easier to maintain

Why C?

C is regarded as the "Mother of Modern Programming Languages."

Many modern programming languages borrow concepts from C.

Examples include:

- C++
- Java
- C#
- Objective-C
- Go
- Rust

Learning C helps students understand:

- Memory management
- Variables
- Data representation
- Functions
- Pointers
- Algorithms

2. The Program Compilation Process

Unlike Python, C programs cannot be executed directly.

A C program must first be translated into machine language.

The compilation process consists of four stages.

Source Code (.c)



Preprocessor



Compiler



Assembler



Linker



Executable Program (.exe)

Stage 1 – Preprocessor

Processes all preprocessor directives.

Example

#include <stdio.h>

The preprocessor inserts the required library before compilation.

Stage 2 – Compiler

The compiler checks:

- syntax errors
- variable declarations
- data types

If errors are found, compilation stops.

Stage 3 – Assembler

Converts assembly language into machine code.

Stage 4 – Linker

Combines your program with required libraries.

Produces the executable file.

Windows

program.exe

Linux

program.out

GCC Compiler

GCC stands for

GNU Compiler Collection

It is one of the world's most widely used C compilers.

It checks syntax and converts C code into executable programs.

Example command:

gcc main.c -o municipal

Run:

Windows

municipal.exe

Linux/macOS

./municipal

Source Code vs Executable

Source Code	Executable
Written by programmer	Generated by compiler
Human readable	Machine readable
Extension .c	.exe or executable
Can be edited	Cannot easily be edited

What is an IDE?

IDE stands for

Integrated Development Environment

It provides tools for software development in one application.

Examples

- Visual Studio Code
- Visual Studio
- Code::Blocks
- Eclipse
- CLion

An IDE usually includes:

- Code editor
- Compiler integration
- Debugger
- Terminal
- Extensions

Installing Visual Studio Code
Step 1

Download Visual Studio Code

https://code.visualstudio.com

Install using the default settings.

Step 2

Open Visual Studio Code.

Step 3

Install the following extensions.

Search for and install:

- C/C++ (Microsoft)
- C/C++ Extension Pack
- Code Runner
- GitLens (optional)
- Error Lens (optional)

Installing GCC Compiler
Windows

Install MSYS2 or MinGW-w64.

Recommended:

https://www.msys2.org/

After installation:

pacman -S mingw-w64-ucrt-x86_64-gcc

Add the GCC **bin** folder to the Windows PATH.

Verify installation:

gcc --version

Linux (Ubuntu)
sudo apt update

sudo apt install build-essential

Verify:

gcc --version

macOS

Install Xcode Command Line Tools:

xcode-select --install

Verify:

gcc --version

Configuring VS Code for C

Create a folder

ProgrammingInPractice

Inside create

Week1

Create

main.c

Open the terminal

Compile

gcc main.c -o municipal

Run

Windows

municipal.exe

Linux/macOS

./municipal

Installing Git

Download Git

https://git-scm.com

Verify installation

git --version

Creating a GitHub Account

Visit

https://github.com

Create an account using your student email.

Recommended username format:

studentnumber-surname

Example

223456789-john

Configuring Git

Open Terminal

Configure your name

git config --global user.name "John Smith"

Configure email

git config --global user.email "student@nust.na"

Verify

git config --list

Creating Your First Repository

Inside your project folder

git init

Check status

git status

Add files

git add .

Commit

git commit -m "Week 1 - First C Program"

Push to GitHub after creating an empty repository online:

git remote add origin https://github.com/username/ProgrammingInPractice.git

git branch -M main

git push -u origin main

Structure of a C Program

```
#include <stdio.h>

int main()
{

    return 0;

}
```

Understanding the Program

#include <stdio.h>

Imports the Standard Input/Output library.

int main()

The entry point of every C program.

Execution always begins here.

{

Marks the beginning of the program block.

}

Marks the end of the program block.

return 0;

Indicates successful program execution.

Header Files

Header files provide reusable functions.

Example

#include <stdio.h>

Provides

- printf()
- scanf()

Other examples

#include <math.h>

#include <string.h>

#include <stdlib.h>

Input and Output

Output
printf("Hello World");

Displays information on the screen.

Input
scanf("%d",&age);

Reads information from the keyboard.

Practical Exercise 1

Write a program that displays:

Municipal Financial Management System

Welcome to Windhoek Municipality

Solution

```
#include <stdio.h>

int main()
{
```

```
    printf("Municipal Financial Management System\n");
    printf("Welcome to Windhoek Municipality\n");

    return 0;
}
```

Practical Exercise 2

Modify the program so that it asks the user for:

- Municipality Name
- Mayor
- Population

and displays the information entered.

Example Output

```
Enter Municipality Name:
Windhoek

Enter Mayor:
John Doe

Enter Population:
486000

--------------------------------

Municipality: Windhoek

Mayor: John Doe

Population: 486000
```

Solution

```
#include <stdio.h>

int main()
{
    char municipality[50];
    char mayor[50];
    int population;

    printf("Municipal Financial Management System\n\n");

    printf("Enter Municipality Name: ");
    scanf("%49s", municipality);
```

```
    printf("Enter Mayor: ");
    scanf("%49s", mayor);

    printf("Enter Population: ");
    scanf("%d", &population);

    printf("\n--------------------------------\n");
    printf("Municipality : %s\n", municipality);
    printf("Mayor        : %s\n", mayor);
    printf("Population   : %d\n", population);

    return 0;
}
```

Note: This simple example uses scanf("%49s", ...), which reads a single word. We will learn how to accept full names with spaces (e.g., "City of Windhoek" or "John Doe") later in the course using safer input techniques.

Summary

By the end of this lesson you should now be able to:

- Explain what programming is.
- Describe the C compilation process.
- Explain the purpose of GCC.
- Differentiate between source code and executables.
- Explain the purpose of an IDE.
- Install and configure Visual Studio Code.
- Install GCC.
- Install Git and create a GitHub account.
- Write, compile and execute a simple C program.
- Use printf() and scanf().
- Upload your first project to GitHub.

Lab Exercise (To Be Submitted)

Develop a program called **Municipal Financial Management System** that:

1. Displays the system title.
2. Displays **"Welcome to Windhoek Municipality"**.
3. Prompts the user to enter:
    - Municipality Name
    - Mayor's Name
    - Population
4. Displays the captured information in a neatly formatted report.

5. Save the program as main.c.
6. Compile it successfully using GCC.
7. Commit the project to GitHub with the commit message:

    Week 1 - Municipal Financial Management System

This lab forms the foundation of the semester-long project, which will be expanded with additional modules such as employee management, budgeting, procurement, assets, and payroll in the coming weeks.

3. References

Free Software Foundation. (2025). *Using the GNU Compiler Collection (GCC): GCC online documentation*. https://gcc.gnu.org/onlinedocs/

Git. (2025). *Git documentation*. https://git-scm.com/doc

GitHub. (2025). *GitHub documentation*. https://docs.github.com/

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton & Company.

Microsoft. (2025). *Visual Studio Code documentation*. https://code.visualstudio.com/docs

Reek, K. A. (2013). *Pointers on C*. Addison-Wesley.

Schildt, H. (2017). *C: The complete reference* (4th ed.). McGraw-Hill Education.

Stallings, W. (2020). *Computer organization and architecture: Designing for performance* (11th ed.). Pearson.

The Linux Foundation. (2025). *The Linux command line*. https://www.linuxfoundation.org/